1. Abstract

Any business that sells something online ends up collecting a list of leads — people who filled a form, asked for a demo, or showed some interest in what the company is offering. The problem is that not all leads are equally serious. Some are close to buying, some are just browsing, and some will probably never convert at all. Most small businesses either go through this list manually or use a simple point-based system to decide who to contact first, and both of these take up a lot of time without being very reliable.

This project tries to make this process a bit smarter using Machine Learning. We take past data about leads — things like how they found the company, how many times they visited the website, whether they had bought something before — and use it to train a model that predicts how likely a new lead is to convert. Based on this prediction, each lead gets a score and is placed into one of three categories: Hot, Warm, or Cold. The system also tries to show which factors led to a particular score, and gives a simple suggestion on what the sales person should do next. All of this is shown together on a dashboard so the sales team can see which leads need attention first.

It is important to mention that this is meant to be a decision-support tool, not a replacement for the sales team. The system only gives suggestions based on patterns in past data — whether to actually contact a lead, and how, is still a decision made by the person handling sales.

2. Introduction & Problem Statement

A lead, in simple words, is a potential customer — someone who has shown some interest but hasn't actually bought anything yet. Businesses get leads from their website, from ads, from social media pages, and sometimes just from a contact form on their site. The more channels a business uses to market itself, the more leads it usually ends up with.

The real problem starts when there are too many leads for the sales team to handle properly. If a company gets even a couple hundred leads in a month, it's not really possible for a small sales team to sit down and study each one carefully. Because of this, many companies just go through leads in whatever order they came in, or they use a point-based system where a lead gets extra points for doing certain things, like requesting a demo or visiting the pricing page. But these points are usually decided once by a person and rarely get updated based on what actually happened to those leads later.

So the question this project is really trying to answer is this — out of all these leads, which ones should the sales team call first, and why? Doing this manually is slow, and it depends a lot on who is reviewing the leads that day. A fixed rule-based system also doesn't really learn anything; it just keeps applying the same rules regardless of outcomes. Our project tries to work around this by using Machine Learning to study historical lead data and find the patterns that separate leads who actually converted from those who didn't.

3. Objectives

- Predict how likely a lead is to convert, using a Machine Learning model trained on past lead data.
- Divide leads into Hot, Warm, and Cold categories based on this prediction.
- Show, in simple terms, which factors mostly influenced a lead's score.
- Suggest a follow-up action for the sales team based on each lead's category.
- Build a dashboard where all of this — score, category, explanation, recommendation — can be seen in one place.
- Try out a few different ML algorithms and compare them properly instead of picking one randomly.

4. Proposed System

Here is roughly how we are planning to build the system, step by step. First, lead information is collected and cleaned up, since raw data usually has missing values or inconsistent formatting. This data is then converted into a form the ML model can actually use. The trained model looks at this data and predicts a conversion probability for each lead. Based on this probability, the lead is marked Hot, Warm, or Cold. Along with this, the system tries to explain why it gave that score, and suggests what the sales person should probably do next. All of this finally shows up on a dashboard.

Module	What it does
Data Collection	Gathers lead details — where the lead came from, how they interacted with the website, past purchases, etc.
Preprocessing	Cleans the data — handles missing values, removes duplicates, converts text fields into numbers, splits data for training and testing.
EDA (Data Analysis)	Looks at the data before modelling, to see which factors seem related to conversion.

Module	What it does
ML Model	Trains and compares a few algorithms — Logistic Regression, Decision Tree, Random Forest, and XGBoost.
Lead Scoring	Gives each lead a conversion probability and a Hot/Warm/Cold label.
Explanation	Shows the main factors behind a given prediction, using feature importance or SHAP.
Recommendation	Gives a simple next-step suggestion based on the lead's category.
Dashboard	Lets the sales team search, filter, and view leads along with their scores and explanations.

Most small businesses that don't use big CRM software right now just rely on manual review or a fixed point system. That works okay when there are only a handful of leads, but it doesn't scale well, and more importantly, it doesn't learn anything from what actually happened to past leads. Our system tries to improve on this a little by training on real historical outcomes instead of relying on rules someone decided once and never updated.

5. System Architecture & Workflow

Lead Data → Preprocessing → Feature Engineering → ML Model → Conversion Probability → Lead Category → Explanation → Recommendation → Dashboard

In our case, the thing we are trying to predict (the target variable) is simply whether a lead converted or not — 1 if they converted, 0 if they didn't. This makes it a binary classification problem, which is one of the more common problems solved using Machine Learning. The model doesn't just give a plain yes or no, though. It gives a probability, something like 0.87, which we show as 87%. Based on this number, the lead gets marked as Hot, Warm, or Cold. We haven't fixed the exact cut-off values for these categories yet — that will be decided later, once we actually test the model on real data.

6. Algorithms & Evaluation Metrics

We are planning to try out four algorithms and compare them, instead of picking just one at the start:

Algorithm	Why we're considering it	Where it might fall short
Logistic Regression	Simple and fast, gives a probability directly, easy to understand	Might not capture complicated, non-linear patterns
Decision Tree	Easy to explain — you can literally read the rules it used	Can overfit if it grows too deep

Algorithm	Why we're considering it	Where it might fall short
Random Forest	Combines many trees, usually more reliable than a single tree	Harder to explain compared to one simple tree
XGBoost	Tends to perform well on this kind of tabular data	Needs more tuning to get right

We are not going to judge the model just by accuracy. Accuracy can be misleading here, because usually far fewer leads convert than don't. If a model just predicted “not converted” every single time, it could still show a high accuracy score while being completely useless for our purpose. So along with accuracy, we will also look at precision, recall, F1-score, the confusion matrix, and ROC-AUC to get a fairer picture of how well the model is actually doing, especially for the leads that do convert.

Along with the score, we also want the system to explain why it gave that score. Instead of just showing “Hot Lead – 87%”, it should say something like “this lead looks promising mainly because they've visited the website often and bought from us before.” We plan to do this using feature importance, and possibly SHAP values if we manage to implement it properly within the project timeline.

7. Technology Stack

Nothing unusual here — mostly tools that are commonly used for this kind of project and that we're already somewhat familiar with from coursework.

Layer	Technology
Language	Python
Data Handling	Pandas, NumPy
Visualization	Matplotlib, Seaborn (Plotly if needed for the dashboard)
Machine Learning	Scikit-learn, XGBoost
Explainability	SHAP / feature importance
Backend	Flask or FastAPI
Frontend	HTML, CSS, JavaScript, with Bootstrap or React
Database	MySQL / PostgreSQL / SQLite

8. Requirements & User Roles

Functional Requirements

- Login for users, adding/importing leads, viewing and searching/filtering the lead list

- Running a prediction on a lead and viewing its score, category, explanation, and recommendation
- Dashboard with basic analytics, exporting results, and a page to check model performance

Non-Functional Requirements

- The system should be easy to use, reasonably fast, and should keep lead data reasonably secure and private.

User Roles

- Admin — manages users and the dataset, and can check how well the model is performing.
- Sales User — views leads, runs predictions, and acts on the recommendations if they find them useful.

9. Expected Output (Just an Example)

To give an idea of what the output might look like — this is not real data, just an example to explain the concept:

Input: Previous Purchase = Yes, Website Visits = 8, Product Interactions = 6, Email Engagement = High

Output: Conversion Probability = 87% → HOT LEAD

Main factors: previous purchase, high website engagement, product interaction.

Suggested action: “Prioritise this lead for a personal follow-up.” The actual numbers we get will depend entirely on the dataset we use and how well the model performs once trained.

10. Advantages, Limitations & Future Scope

Advantages

- Saves the sales team some time, since they don't have to review every single lead manually.
- Gives a consistent score instead of relying purely on individual judgement.
- Tries to explain its predictions instead of just giving a plain number.

Limitations

- The predictions are only as good as the dataset used to train the model — if the data is limited or biased, the results will reflect that.
- Patterns from the past don't always hold true in the future, since customer behaviour can change.
- These are probability-based predictions, not guarantees — a Hot lead can still choose not to convert.
- Connecting this to a real company's CRM system would need more work than what is covered in this project.

Future Scope

- Connecting the system to actual CRM tools or sending automatic email/WhatsApp follow-ups.
- Retraining the model regularly as new lead data comes in.
- Extending the idea to predict things like customer churn or lifetime value.

11. Feasibility & Development Methodology

This project is achievable within our timeline mainly because everything we're using — Python, Scikit-learn, XGBoost, SHAP — is free, open-source, and reasonably well documented. We don't need any special hardware either; a normal laptop is enough to train and test these models. From a usage point of view too, it stays simple: a sales person just logs in, looks at the dashboard, and decides what to do with the suggestions — the system doesn't take over any part of the actual selling.

We're planning to build this in stages rather than all at once — starting with requirement analysis and dataset preparation, then moving to data analysis, model building, evaluation, and finally the backend, dashboard, and integration, followed by testing and writing up the documentation.

12. Deliverables & Conclusion

- A working web application with the trained ML model connected to it
- A short report comparing how the different models performed
- The dashboard, along with the project documentation and a final demo

To sum up, this project combines a few things we've studied separately — Machine Learning, data analysis, and web development — into one working system that tries to solve a real, practical

problem. It won't replace a sales team, and it won't guarantee any sale either. What it can do is help the team spend their time on the leads that are actually worth following up with, based on patterns learned from past data instead of just a gut feeling.

13. References

- Standard Machine Learning textbooks covering classification and model evaluation (exact edition to be finalised).
- Official Scikit-learn documentation (scikit-learn.org)
- Official XGBoost documentation (xgboost.readthedocs.io)
- Official SHAP documentation (shap.readthedocs.io)
- Relevant papers on lead scoring and explainable AI, to be added after our literature review is done.

One-Minute Viva Explanation

"Our project is called LeadIQ. The idea is simple — businesses get a lot of leads, but their sales team doesn't have time to check every single one manually. So we use Machine Learning to look at past lead data and learn what usually separates a lead that converts from one that doesn't — things like how many times they visited the website, or whether they've bought something before. Using this, our model predicts how likely a new lead is to convert, and we label it as Hot, Warm, or Cold. We also try to explain why the model gave that score, and give a simple suggestion on what to do next, like prioritising a call for a Hot lead. All of this is shown on a dashboard. It's not meant to replace the sales team — it's just there to help them figure out where to focus first."

Project in One Sentence

LeadIQ uses Machine Learning to predict how likely a lead is to convert, sorts it into Hot, Warm, or Cold, explains why, and suggests what to do next.

Problem → Solution → ML → Output

- Problem: Too many leads, not enough time to check each one manually.
- Solution: Use past lead data to automatically figure out which leads are worth prioritising.
- ML: Train and compare a few algorithms (Logistic Regression, Decision Tree, Random Forest, XGBoost) to predict conversion probability.

- Output: A score, a Hot/Warm/Cold label, an explanation, and a suggested next step — shown on a dashboard.